Name: B.Siddhu
Reg No: 127003037

Big Data in Industry

Take-Home Assignment 1: Weekly Business Review for an E-commerce Marketplace

Dataset: Olist Brazilian E-Commerce (Kaggle), 99,441 orders, Sep-2016 to Oct-2018. Tools: Python, DuckDB, SQL, Airflow, matplotlib.

Summary

- **Pipeline:** 9 CSVs, raw, staging, mart in DuckDB. Idempotent, logged, 8 + 3 SQL quality checks with alarms, and a daily Airflow DAG.
- **Data quality:** keys and joins are clean. Fixed or flagged 166 impossible carrier dates, 366 payment mismatches, 547 duplicate reviews and 14 status/date conflicts.
- **Growth:** GMV rose from R$137k (Jan-17) to about R$1.0-1.16M per month, flat since Mar-18. Repeat-purchase rate is only 3.04%.
- **Key finding:** late deliveries (6.8% of orders) score 2.27 vs 4.29 stars. Late plus undelivered orders cause 43% of all 1-2 star reviews.
- **Recommendation:** set delivery promises from recent route data. Back-tested: 21% fewer late orders.

Part A - Ingestion Pipeline (ELT)

The pipeline (`pipeline/run_pipeline.py`) loads the nine Olist CSVs into a single DuckDB warehouse and builds three layers. Each step is logged with start, end, duration and row counts to the console and `logs/pipeline.log`.

Step / layer	What it does	Output (rows)
1. validate_inputs	Checks all 9 files exist and every required column is present (column list per file in `pipeline/config.py`).	-
2. raw.*	CSV loaded as-is (all text) + `_source_file`, `_loaded_at`.	orders 99,441; items 112,650; payments 103,886; reviews 99,224
3. dq_gate	8 SQL data-quality checks on raw data (Part B). Alarm stops the run before anything downstream is rebuilt.	dq.latest_gate
4. staging.*	Typed columns, trimmed text, de-duplication and the DQ fixes (one review per order, invalid carrier dates nulled, flags added).	7 tables; reviews 98,673 (1 per order)
5. mart.fct_orders	**One row per order**: GMV, freight, payment total, item and seller count, all delivery dates, promised days, delay days, review score, flags.	99,441
mart.fct_order_items	One row per order line with category, seller and buyer state.	112,650
6. dq_post	3 checks on the finished mart: grain, revenue reconciliation, review uniqueness.	dq.latest_post

- **Idempotent:** every table is rebuilt with `CREATE OR REPLACE` inside a transaction, so re-running replaces rather than appends. Verified by running twice: every table has identical row counts and mart GMV is unchanged. The only append-only table is the audit log `dq.check_history`, keyed by run_id.
- **No double counting:** items and payments are each aggregated to order level in separate CTEs before joining, so all mart joins are 1:1. Check POST01 enforces this on every run.
- **Failure handling:** each failure type has its own exception (MissingFileError, SchemaError, DataQualityError). Tested: deleting a CSV, renaming the `price` column, and injecting a duplicate order each stopped the pipeline with exit code 1 and a message naming the file, column or check. A failing SQL step is rolled back.
- **Bonus - Airflow:** `dags/olist_wbr_dag.py` runs the six steps as six tasks, `schedule="@daily"`, catchup off, one active run (DuckDB has a single writer), 1 retry. A DQ alarm fails its task and blocks staging and marts. Tested with `airflow dags test` on Airflow 3.3: all six tasks succeeded.
- **Run:** `pip install -r requirements.txt` → put CSVs in `data/raw/` → `python -m pipeline.run_pipeline` (full steps in README.md).

Name: B.Siddhu
Reg No: 127003037

Part B - Data Quality Checks

Eight checks run on the raw layer before anything is built, and three more on the finished mart. Each check is a `.sql` file returning *failing_rows* and *total_rows*; its header sets an allowed failure ratio. If any check exceeds it, the pipeline raises `DataQualityError` ("DATA QUALITY ALARM") and stops. Key integrity checks allow zero failures; known, explainable source issues get a small tolerance, so a sudden jump still raises the alarm. Results from the final run:

ID	Check (SQL in sql/dq/)	Failing / total rows	Alarm if over	Handling	Why
DQ01	Null or duplicate order_id in orders	**0** / 99,441	0 rows	Hard stop (0 allowed)	order_id is the grain of the mart.
DQ02	Delivered-to-customer date earlier than purchase date	**0** / 99,441	0 rows	Hard stop (0 allowed)	Would make delay metrics meaningless.
DQ03	Carrier hand-over date earlier than purchase date	**166** / 99,441	1.0%	**Fixed:** carrier date set to NULL, flagged	Impossible timestamp; the rest of the order is valid, so keep the order.
DQ04	Payment total differs from item price + freight (> R$0.01)	**366** / 98,665	1.0%	**Flagged** (is_payment_mismatch), kept	269 of 366 pay MORE than items+freight (instalment interest). Revenue uses price+freight, so these orders are still correct.
DQ05	Orders with no matching customer record	**0** / 99,441	0 rows	Hard stop (0 allowed)	Order could not be attributed to a customer or state.
DQ06	*(own)* Status contradicts delivery date	**14** / 99,441	0.1%	**Flagged**, excluded from delay metrics	Status and dates disagree, so the delay cannot be trusted.
DQ07	*(own)* Order has more than one review	**547** / 98,673	1.0%	**Fixed:** keep latest answered review	The last answer is the customer's final opinion; avoids double-weighting.
DQ08	*(own)* Order items with no parent order or invalid price	**0** / 112,650	0 rows	Hard stop (0 allowed)	Orphan or zero-price items would corrupt revenue.

Rows with a non-zero failing count are the real issues found in the data. Post-mart checks: POST01 mart.fct_orders has exactly one row per staged order - 0 failing; POST02 Mart revenue reconciles with staged order items - 0 failing; POST03 Staged reviews are unique per order and scores in 1-5 - 0 failing.

Other observations (not errors): 775 orders have no items (767 of them canceled or unavailable) and 1 order has no payment. They are kept in the mart with GMV 0 so order counts stay complete. 610 products have no category and are labelled 'unknown'. Money is cast to DECIMAL, not float. With floats DQ04 reported 443 mismatches instead of 366, all of them rounding noise.

Part C - SQL at Scale (CTEs + window functions; files in sql/analysis/)

Month	GMV	MoM %		Month	GMV	MoM %
2017-01	R$136,943	n/a*		2017-11	R$1,173,387	+53.4%
2017-02	R$283,562	+107.1%		2017-12	R$862,300	-26.5%
2017-03	R$425,618	+50.1%		2018-01	R$1,102,475	+27.9%
2017-04	R$405,849	-4.6%		2018-02	R$980,155	-11.1%
2017-05	R$582,711	+43.6%		2018-03	R$1,152,785	+17.6%
2017-06	R$499,652	-14.3%		2018-04	R$1,156,635	+0.3%
2017-07	R$578,754	+15.8%		2018-05	R$1,146,158	-0.9%
2017-08	R$661,986	+14.4%		2018-06	R$1,020,606	-11.0%
2017-09	R$717,103	+8.3%		2018-07	R$1,039,817	+1.9%
2017-10	R$764,861	+6.7%		2018-08	R$997,040	-4.1%

Q1. Month-over-month revenue (LAG). GMV = price + freight of non-canceled orders. A month calendar is generated first, so LAG always compares with the previous *calendar* month (Nov-2016 has no orders). Months before 2017 and after Aug-2018 have almost no orders and are left out of the table. *Jan-17 growth is not meaningful (Dec-16 had 1 order).

GMV grew from R$136,943 (Jan-17) to around R$1.0-1.16M a month in 2018. The peak was 2017-11 (R$1,173,387, Black Friday). Growth has been flat since Mar-2018.

Q2. Top 3 categories per customer state (DENSE_RANK over PARTITION BY state). The query returns all 27 states (81 rows, because ties are kept). The five largest states are shown below. Health & beauty, watches & gifts and bed/bath/table lead almost everywhere.

State	#1 category (GMV)	#2 category (GMV)	#3 category (GMV)
SP	bed bath table (R$555,664)	health beauty (R$517,654)	watches gifts (R$458,579)
RJ	watches gifts (R$197,756)	bed bath table (R$179,896)	health beauty (R$164,189)
MG	health beauty (R$178,458)	bed bath table (R$157,332)	watches gifts (R$134,331)
RS	bed bath table (R$72,714)	furniture decor (R$67,648)	sports leisure (R$63,415)
PR	sports leisure (R$67,648)	watches gifts (R$65,072)	health beauty (R$62,738)

Q3. 7-day rolling average of daily orders (AVG OVER ROWS 6 PRECEDING). A day calendar is built first so days with no orders count as zero; the first 6 days are NULL. The rolling average rose from about 6/day (Jan-17) to 200-300/day in 2018. The peak was 494/day in the week ending 29-Nov-2017 (Black Friday). See Fig 2.

Q4. Repeat-purchase rate. `customer_id` is created new for every order, so counting by it gives **0.0%**. The real person is `customer_unique_id`: 2,888 of 94,990 customers bought twice or more, a repeat rate of **3.04%** (1.034 orders per customer). Almost all of Olist's growth comes from new customers.

Q5. Delivery delay vs estimated date, and review score. delay = delivered date - estimated date. On average orders arrive **11.9 days early**, but the relationship is a cliff, not a slope. Correlation of delay with score is r = -0.27, yet the score drops by about one star on the first late day (3.29) and below 2.1 once an order is 4 or more days late. Being 15+ days early scores almost the same as being just on time.

Delay vs promised date	Orders	% of orders	Avg delay (days)	Avg review	% 1-2 star
15+ days early	34,771	36.3%	-20.6	4.32	9.0%
8-14 days early	36,158	37.7%	-11.3	4.31	8.9%
0-7 days early	18,514	19.3%	-4.5	4.19	10.4%
1-3 days late	1,852	1.9%	+1.8	3.29	32.1%
4-7 days late	1,748	1.8%	+5.5	2.10	67.7%
8-14 days late	1,446	1.5%	+10.6	1.67	80.2%
15+ days late	1,335	1.4%	+29.0	1.72	78.4%

Part D - Metrics (North Star and supporting / guardrail metrics)

Values are for Jan-2017 to Aug-2018 (87 full weeks). Weekly versions are produced by `sql/metrics/weekly_metrics.sql` for the Weekly Business Review.

Metric	Exact formula (from mart.fct_orders)	Value	Why it matters
North Star: On-time Happy Orders / week	COUNT(orders WHERE is_delivered AND delivered_date <= estimated_date AND review_score >= 4) per purchase week	848/wk (74% of orders)	Counts only orders that were sold, kept their promise and pleased the customer. It grows only if volume AND experience grow together.
GMV / week (supporting)	SUM(price + freight) over order items WHERE status NOT IN (canceled, unavailable)	R$180,326/wk	Top-line scale of the marketplace; seller and commission revenue follow it.
On-time delivery rate (supporting, driver)	COUNT(delivered AND delivered_date <= estimated_date) / COUNT(delivered)	93.2%	The strongest controllable driver of review score (Fig 3).
1-2 star review share (guardrail)	COUNT(review_score <= 2) / COUNT(orders with a review)	14.6%	Early warning of churn and seller-quality problems. Must not rise while GMV is pushed.
Cancel / unavailable rate (guardrail)	COUNT(status IN (canceled, unavailable)) / COUNT(orders)	1.19%	Catches stock-outs and seller failures; a sale that never ships is the worst experience of all.

Repeat-purchase rate (Q4, 3.04%) is tracked monthly as a long-term guardrail; it is too small and slow-moving for a weekly review.

Weekly Business Review snapshot - last 8 full weeks

This is the table leadership would see each Monday (output of `weekly_metrics.sql`; weeks from 20-Aug-2018 are incomplete because the dataset ends, so they are excluded). The weakest week for on-time delivery was 30-Jul (90.8%), and its 1-2 star share was 13.9%. The guardrail moves with the delivery metric, as the memo argues.

Week starting	Orders	North Star (on-time happy)	NS WoW	GMV	On-time rate	1-2 star share	Cancel rate
25-Jun-2018	1,401	1,153	-	R$222,960	99.3%	10.6%	0.57%
02-Jul-2018	1,196	993	-13.9%	R$186,189	98.6%	8.8%	0.67%
09-Jul-2018	1,001	747	-24.8%	R$183,505	94.7%	14.6%	1.30%
16-Jul-2018	1,678	1,362	+82.3%	R$260,502	97.0%	11.0%	0.72%
23-Jul-2018	1,646	1,328	-2.5%	R$283,029	97.3%	10.3%	0.73%
30-Jul-2018	2,058	1,537	+15.7%	R$323,756	90.8%	13.9%	1.36%
06-Aug-2018	1,988	1,508	-1.9%	R$324,840	91.5%	12.2%	1.36%
13-Aug-2018	1,875	1,527	+1.3%	R$287,167	96.4%	8.5%	1.23%

Name: B.Siddhu
Reg No: 127003037

Part D - Charts

All charts are generated by `report/make_charts.py` from the same SQL files as Part C (window: Jan-2017 to Aug-2018, the full months with meaningful volume).

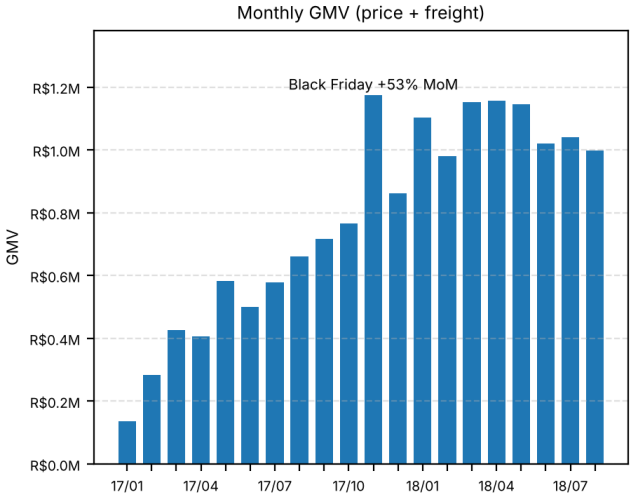


Figure 1. GMV grew about 8x during 2017, then flattened at roughly R$1.0-1.16M a month from Mar-2018. Black Friday (Nov-17) was the single biggest month.

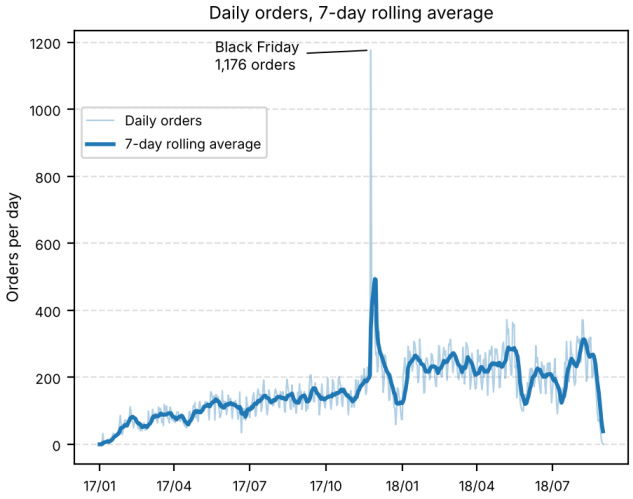


Figure 2. The rolling average removes day-of-week noise. Demand doubled from about 100 to 200-300 orders a day; the Black Friday spike was about 6x a normal day. The drop at the right edge is where the dataset ends.

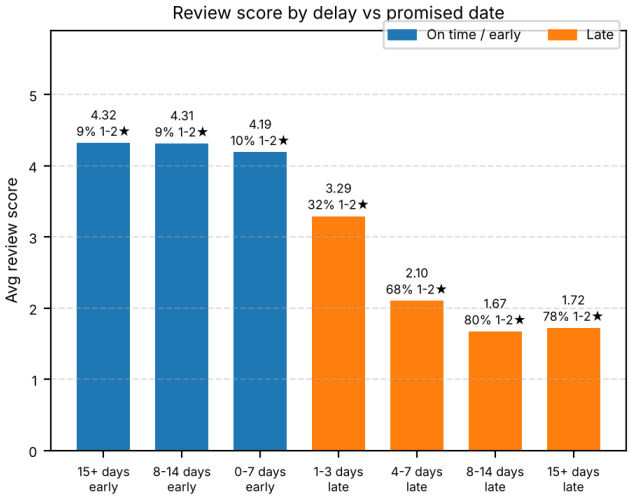


Figure 3. Scores fall off a cliff at the promised date, not gradually with speed. Arriving early earns little extra; arriving late costs one to two and a half stars.

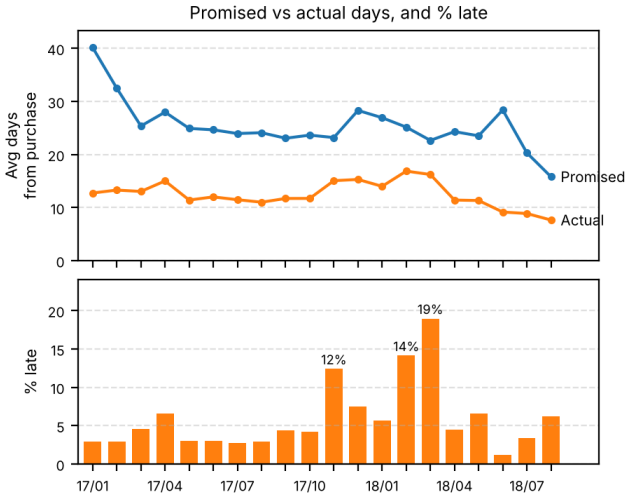


Figure 4. Late rates spike (Nov-17, Feb-Mar-18) when actual delivery time rises while the promise stays fixed or shrinks. In Mar-18 the promise was cut to 23 days just as deliveries slowed to 16.

Name: B.Siddhu
Reg No: 127003037

Part D - Memo to the Vice President

To: Vice President, Operations & Customer Experience **Date:** 12 October 2026
From: B.Siddhu (Reg No: 127003037), Data Analyst
Subject: Broken delivery promises are the biggest cause of bad reviews; set promises from real route data

---

Key finding. Customer satisfaction is lost when we break a delivery promise. Only 6.8% of delivered orders arrive after the estimated date, but those orders average **2.27 stars vs 4.29**, and 61% of them get a 1-2 star review (vs 9% of on-time orders). Late and never-delivered orders are 10% of orders but cause **43% of all 1-2 star reviews**.

Evidence

- **The promise matters, not speed.** Orders 15+ days early score 4.32; orders 0-7 days early score 4.19. One day late drops the score to 3.29, and 4+ days late to about 2 (Fig 3). Customers judge us against the date we showed them.
- **Lateness is not random. It spikes when demand or carriers are stretched.** The late rate was 12% in Nov-17 (Black Friday), 14% in Feb-18 and 19% in Mar-18, against a usual 3-7%. In Mar-18 actual delivery time rose to 16 days while the promise was *cut* to 23 days (Fig 4). Rio de Janeiro reached 34% late in Feb-Mar 2018.
- **The problem sits on specific routes and in the carrier leg.** North-east states run 12-21% late, against 7% overall. For late orders, median carrier-to-door time is 26 days vs 7 for on-time ones. Seller hand-over differs much less (3.0 vs 1.8 days).
- **The current estimate is static.** It does not react when a route slows down, so customers keep getting dates the network cannot meet.

Recommendation. Replace the static estimated delivery date with a **route-level promise**. For every seller-state to customer-state route, the date shown at checkout should be at least the **90th percentile of actual delivery days on that route over the last 28 days** (used when the route has 20 or more recent deliveries). It is recomputed every day by this pipeline. Pilot it on the Rio de Janeiro and north-east routes for 4 weeks, owned by the Logistics product team.

Expected impact and how we will measure it

- **Back-test** (`sql/analysis/x1_promise_backtest.sql`, 93,800 delivered orders, Mar-17 to Aug-18, using only data available at purchase time): late orders fall **21%** (late rate 6.9% to 5.5%), and about **649 orders that got a late-delivery 1-2 star review would have arrived on time**. The cost: 24% of orders show a longer promise, by 5.6 days on average.
- **Success metric:** on-time delivery rate on pilot routes above 95% every month, and a falling 1-2 star share.
  Guardrail: checkout conversion and GMV on pilot routes, because a longer promise can cost sales. Stop the pilot if conversion drops by more than 2%.
- **Honest limit:** a better promise removes about a fifth of late orders, not all. The remaining delays are carrier failures during spikes. The next step is a carrier SLA review on the routes where the 90th percentile keeps rising above the promise; the same pipeline can flag those routes weekly.

Data note: orders after Aug-2018 are incomplete, and late orders from the final weeks may not be delivered yet, so recent late rates are slightly understated.